

# Sistemi Operativi 1

AA 2024/2025

Componenti di un sistema operativo

# Componenti di un S.O.

- Gestione dei processi
- Gestione della memoria primaria
- Gestione della memoria secondaria
- Gestione dell'I/O
- Gestione dei file
- Protezione
- Rete
- Interprete dei comandi

# Gestione dei Processi

# Gestione dei Processi

- Processo = programma in esecuzione
  - Necessità di risorse
  - Eseguito in modo sequenziale un'istruzione alla volta
  - Processi del S.O. vs. processi utente
- Il S.O. è responsabile della
  - Creazione e distruzione di processi
  - Sospensione e riesumazione di processi
  - Fornitura di meccanismi per la sincronizzazione e la comunicazione tra processi

# Gestione della Memoria Primaria

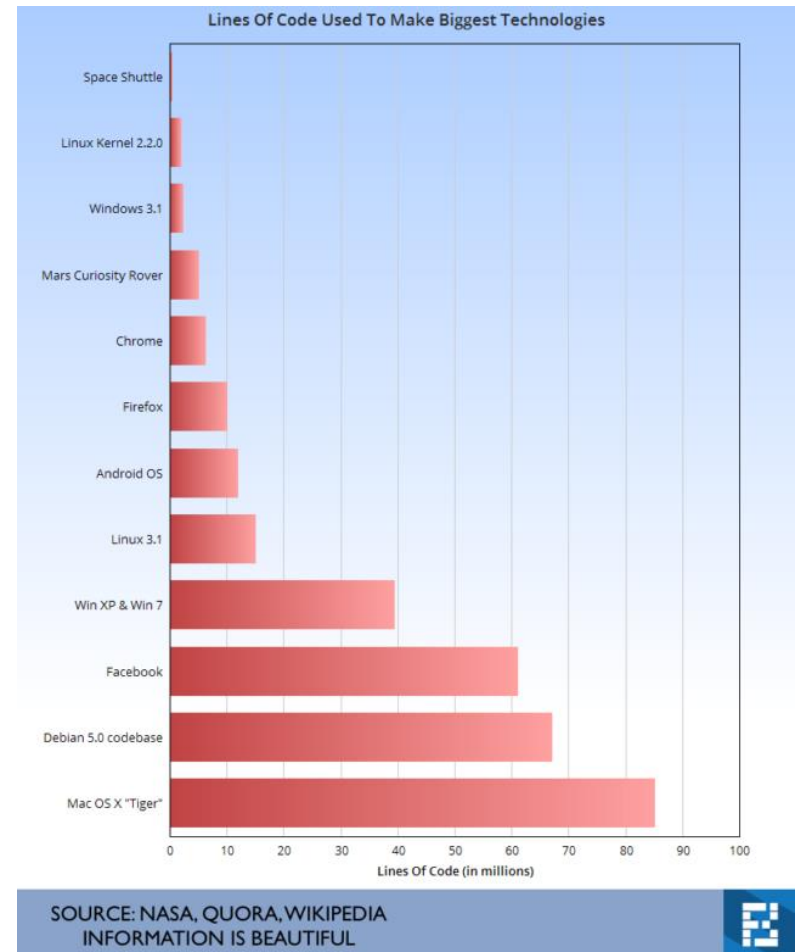
- Memoria primaria conserva dati condivisi dalla CPU e dai dispositivi di I/O
  - Un programma deve essere caricato in memoria per poter essere eseguito
- Il S.O. è responsabile della
  - Gestione dello spazio di memoria (quali parti e da chi sono usate)
  - Decisione su quale processo caricare in memoria quando esiste spazio disponibile
  - Allocazione e rilascio dello spazio di memoria

# Gestione della Memoria Secondaria

- Memoria primaria è volatile e “piccola”
  - Indispensabile memoria secondaria per mantenere grandi quantità di dati in modo permanente
- Tipicamente uno o più dischi (magnetici)
- Il S.O. è responsabile della
  - Gestione dello spazio libero su disco
  - Allocazione dello spazio su disco
  - Scheduling degli accessi su disco

# Gestione dell'I/O

- Il S.O. nasconde all'utente le specifiche caratteristiche dei dispositivi di I/O
- Il sistema di I/O consiste di
  - Un sistema per accumulare gli accessi ai dispositivi (buffering)
  - Una generica interfaccia verso i device driver
  - Device driver specifici per alcuni dispositivi



# Gestione dei File

- Le informazioni sono memorizzate su supporti fisici diversi (dischi, DVD, memory-stick, ...) controllati da driver con caratteristiche diverse
- **File** = astrazione logica per rendere conveniente l'uso della memoria non volatile
  - Raccolta di informazioni correlate (dati o programmi)
- Il S.O. è responsabile della
  - Creazione e cancellazione di file e directory
  - Supporto di primitive per la gestione di file e directory (copia, sposta, modifica, ...)
  - Corrispondenza tra file e spazio fisico su disco
  - Salvataggio delle informazioni a scopo di backup



# Protezione

- Meccanismo per controllare l'accesso alle risorse da parte di utenti e processi
- Il S.O. è responsabile della:
  - Definizione di accessi autorizzati e non
  - Definizione dei controlli da imporre
  - Fornitura di strumenti per verificare le politica di accesso

# Sistemi Distribuiti

- Sistema distribuito = collezione di elementi di calcolo che non condividono né la memoria né un clock
  - Risorse di calcolo connesse tramite una rete
- Il S.O. è responsabile della gestione “in rete” delle varie componenti
  - Processi distribuiti
  - Memoria distribuita
  - File system distribuito
  - ...

# Come usare i servizi del S.O.?

- Il S.O. implementa le funzionalità vista mediante servizi chiamate ***system calls***
- *Come possono essere usati questi servizi?*

# Esempio di system calls

file di origine

file di destinazione

Esempio di ciclo esecutivo di una chiamata di sistema

- Acquisisce il nome del file in ingresso
  - Scriva messaggio di richiesta sullo schermo
  - Accetta i dati in ingresso
- Acquisisce il nome del file in uscita
  - Scriva messaggio di richiesta sullo schermo
  - Accetta i dati in ingresso
- Apri il file in ingresso
  - Se il file non esiste, termina con errore
- Crea il file in uscita
  - Se il file esiste, termina con errore
- Ripete
  - Legge dal file in ingresso
  - Scriva sul file in uscita
- Finché c'è ancora da leggere
- Chiude il file in uscita
- Scriva messaggio sullo schermo per informare del completamento
- Termina senza errori

# strace

```
execve("/usr/bin/ls", ["ls", "/"], 0x7ffe80d1898 /* 88 vars */) = 0
brk(NULL) = 0x576c6209d000
access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=227923, ...}) = 0
mmap(NULL, 227923, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7bca64d1d000
close(3) = 0
openat(AT_FDCWD, "/usr/lib/libc.so.2", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0ps\0\0\0\0\0"..., 832) = 832
fstat(3, {st_mode=S_IFREG|0755, st_size=42992, ...}) = 0
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7bca64d1b000
mmap(NULL, 45128, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7bca64d0f000
mmap(0x7bca64d12000, 20480, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x3000) = 0x7bca64d12000
mmap(0x7bca64d17000, 8192, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x8000) = 0x7bca64d17000
mmap(0x7bca64d19000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x9000) = 0x7bca64d19000
close(3) = 0
openat(AT_FDCWD, "/usr/lib/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220^\2\0\0\0\0"..., 832) = 832
pread64(3, "\6\0\0\0\4\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0"..., 784, 64) = 784
fstat(3, {st_mode=S_IFREG|0755, st_size=1948952, ...}) = 0
pread64(3, "\6\0\0\0\4\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0"..., 784, 64) = 784
mmap(NULL, 1973104, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7bca64b2d000
mmap(0x7bca64b51000, 1421312, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x24000) = 0x7bca64b51000
mmap(0x7bca64cac000, 348160, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x17f000) = 0x7bca64cac000
mmap(0x7bca64d01000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1d3000) = 0x7bca64d01000
mmap(0x7bca64d07000, 31600, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7bca64d07000
close(3) = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7bca64b2a000
arch_prctl(ARCH_SET_FS, 0x7bca64b2a740) = 0
set_tid_address(0x7bca64b2aa10) = 13326
set_robust_list(0x7bca64b2aa20, 24) = 0
rseq(0x7bca64b2b060, 0x20, 0, 0x53053053) = 0
mprotect(0x7bca64d01000, 16384, PROT_READ) = 0
mprotect(0x7bca64d19000, 4096, PROT_READ) = 0
mprotect(0x576c603ec000, 8192, PROT_READ) = 0
mprotect(0x7bca64d88000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
munmap(0x7bca64d1d000, 227923) = 0
prctl(PR_CAPBSET_READ, CAP_MAC_OVERRIDE) = 1
prctl(PR_CAPBSET_READ, 0x30 /* CAP_??? */) = -1 EINVAL (Invalid argument)
prctl(PR_CAPBSET_READ, CAP_CHECKPOINT_RESTORE) = 1
prctl(PR_CAPBSET_READ, 0x2c /* CAP_??? */) = -1 EINVAL (Invalid argument)
prctl(PR_CAPBSET_READ, 0x2a /* CAP_??? */) = -1 EINVAL (Invalid argument)
prctl(PR_CAPBSET_READ, 0x29 /* CAP_??? */) = -1 EINVAL (Invalid argument)
getrandom("\x0e\xbb\xcc\x6\xbb9\x64\x4f\xd6\x2e", 8, GRND_NONBLOCK) = 8
brk(NULL) = 0x576c6209d000
brk(0x576c620be000) = 0x576c620be000
openat(AT_FDCWD, "/usr/lib/locale/locale-archive", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=3055776, ...}) = 0
mmap(NULL, 3055776, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7bca64800000
close(3) = 0
openat(AT_FDCWD, "/usr/share/locale/locale.alias", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=2998, ...}) = 0
read(3, "# Locale name alias data base.\n"..., 4096) = 2998
read(3, "", 4096) = 0
close(3) = 0
openat(AT_FDCWD, "/usr/lib/locale/it_IT.UTF-8/LC_MEASUREMENT", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/lib/locale/it_IT.utf8/LC_MEASUREMENT", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/lib/locale/it_IT/LC_MEASUREMENT", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/lib/locale/it.UTF-8/LC_MEASUREMENT", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/lib/locale/it.utf8/LC_MEASUREMENT", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
```



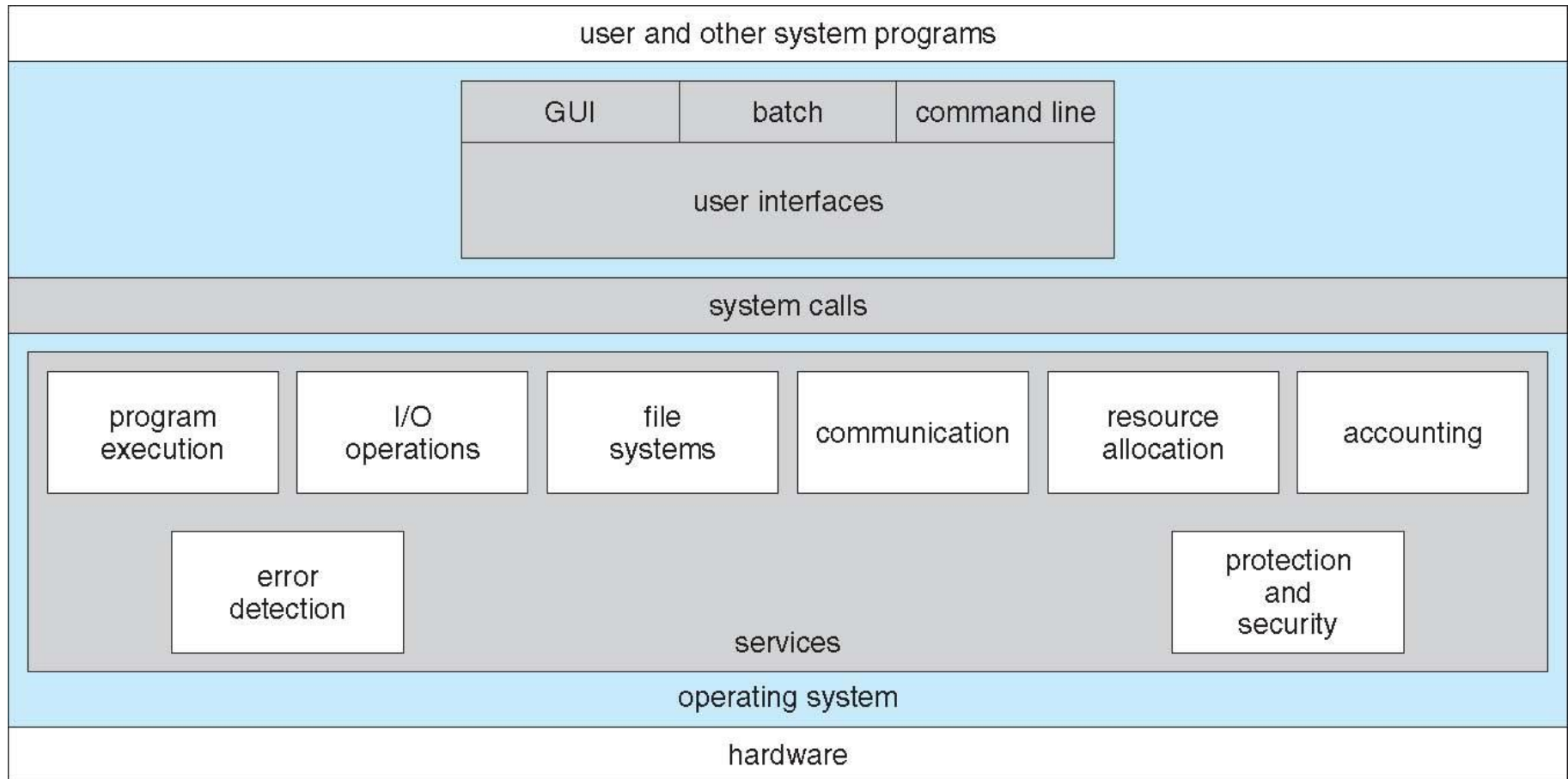
# Interprete dei comandi (Shell)

- La maggior parte dei comandi vengono forniti dall'utente al S.O. tramite “istruzioni di controllo” che permettono di:
  - Creare e gestire processi
  - Gestire l'I/O
  - Gestire il disco, la memoria, il file system
  - Gestire le protezioni
  - Gestire la rete
- Il programma che legge ed interpreta questi comandi è l'interprete dei comandi
  - Funzione: leggere ed eseguire la successiva istruzione di controllo (comando)

# Interprete

- Principalmente ha il compito di prendere i comandi specificati dall'utente e di eseguirli
  - Codice comandi nell' interprete
  - Codice comandi in programmi predefiniti, da riferire semplicemente con il nome del programma (es. rm)
    - Se sono predefiniti per cambiare la semantica dei comandi non è necessario modificare la shell
    - Soltanto l'aggiunta di nuovi comandi comporta la modifica dell'interprete.

# Interfacce Utente di SO





# Interfaccia utente

- L'interprete puo' fornire diversi tipo di interfaccia utente
  - Command-Line (CLI), Graphics User Interface (GUI), Batch
- CLI permette di digitare direttamente comandi testo (es. UNIX)
  - Spesso implementata nel kernel, alcune volte come programma di sistema
  - Spesso è implementata in diverse varianti – **shells** (c, bourne, etc.)

# Interfacce Utente di SO

- La metafora del **desktop** come interfaccia utente
  - Generalmente legata a mouse, tastiera e schermo
  - **Icone** rappresentano i file, programmi, azioni, etc
  - I vari tasti del mouse puntati sull' oggetto visualizzato nell' interfaccia possono eseguire diverse azioni (fornire informazioni, opzioni, eseguire funzioni, aprire directory (noti come **folder**))

# System Call

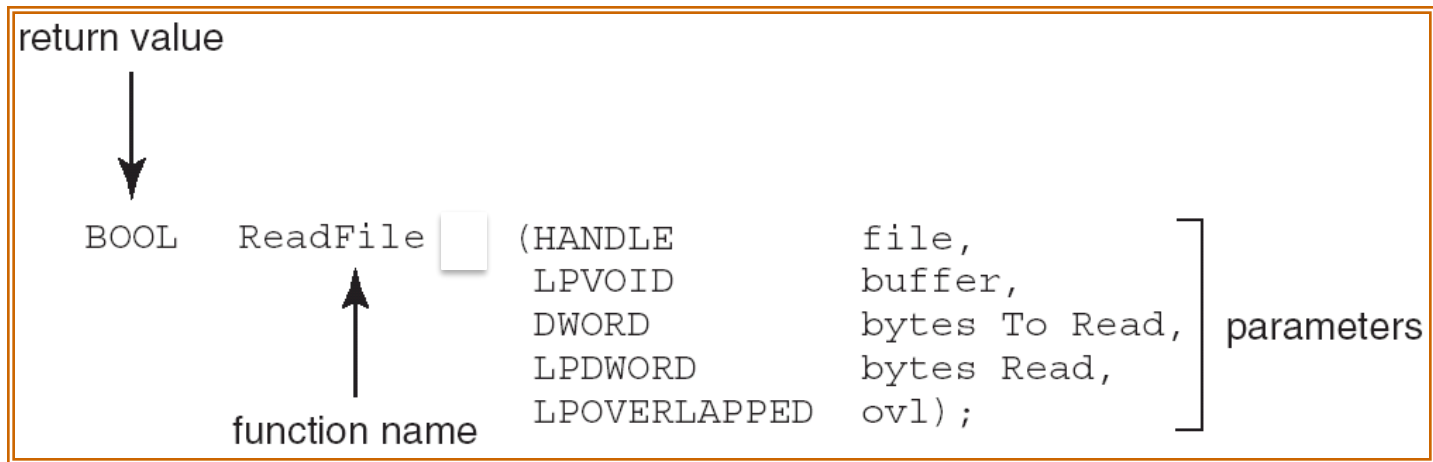
- L'utente usa la shell, ma i processi?
  - Le system calls vengono usate tramite un interfaccia offerta dal S.O. tra i processi e i servizi offerti dal S.O. Application programming interface (API).
- Tipicamente scritte in linguaggio di alto livello (C o C++), qualcuna anche in assembler

# System Call

- Per mascherare i dettagli implementativi il S.O. fornisce un livello intermedio.
- Per lo più chiamate da programmi attraverso l'interfaccia per la programmazione di applicazioni (*Application Program Interface (API)*) di alto livello piuttosto che usate direttamente

# Tipico esempio di API

- Consideriamo la funzione ReadFile()
- Win32 API— la funzione per leggere da un file



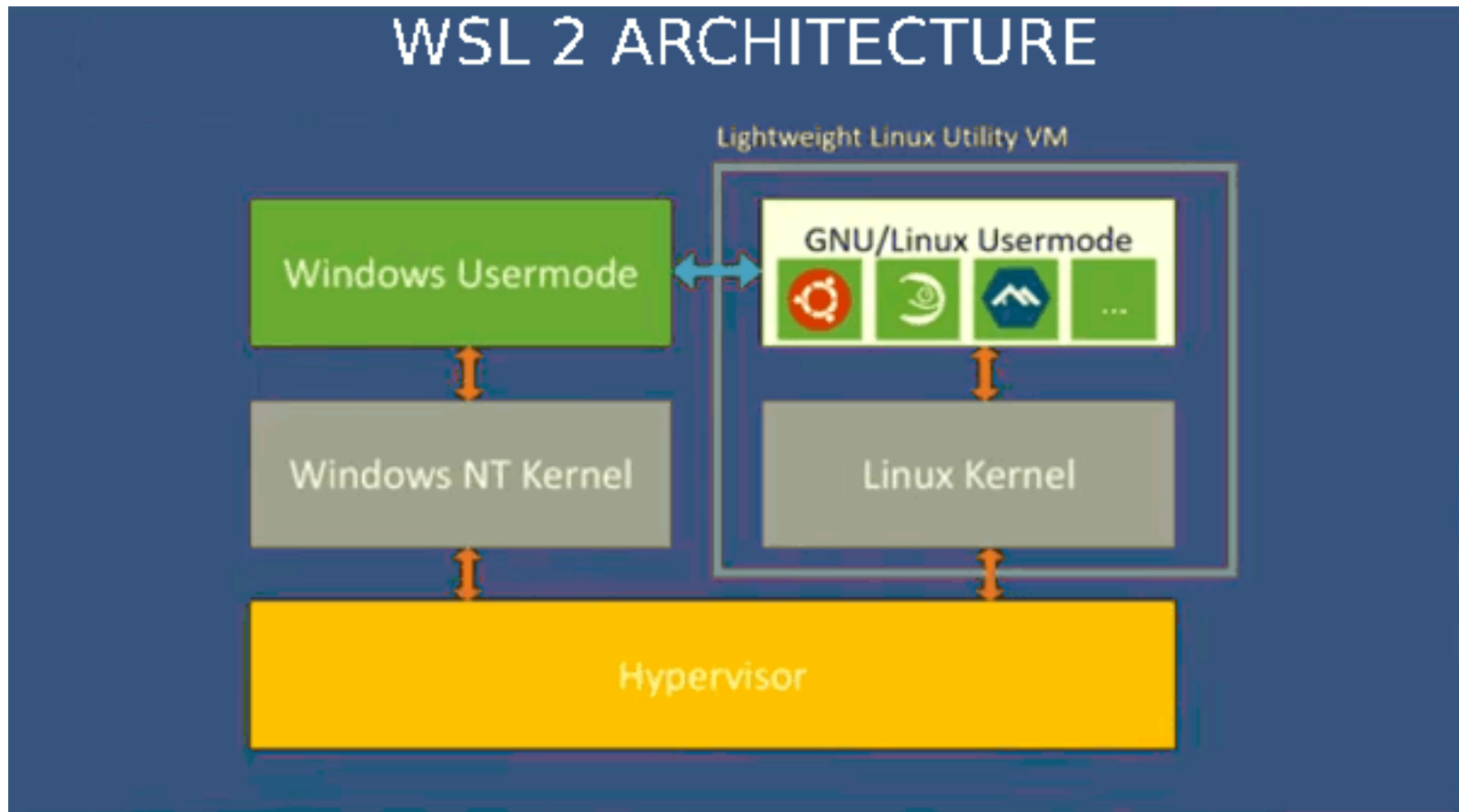
# Tipico esempio di API

- Descrizione dei parametri passati alla `ReadFile()`
  - **HANDLE** file— il file da leggere
  - **LPVOID** buffer—un buffer di interposizione tra letture/scritture
  - **DWORD** bytesToRead—il numero di bytes da leggere dal buffer
  - **LPDWORD** bytesRead—il numero di bytes letti nell' ultima read
  - **LPOVERLAPPED** ovl—indica se è stato usato spooling

# API

- Le 2 APIs piú comuni sono Win32/Win64 API per Windows, POSIX API per sistemi POSIX (Portable Operating-System Interface for Unix)-based (che includono di fatto tutte le versioni di UNIX, Linux, e Mac OS X).
- Dovrebbero garantire portabilità delle applicazioni (almeno sulle stesso tipo di API)

# Running Linux on Windows



Windows Subsystem Linux



# Running Windows on Posix Systems



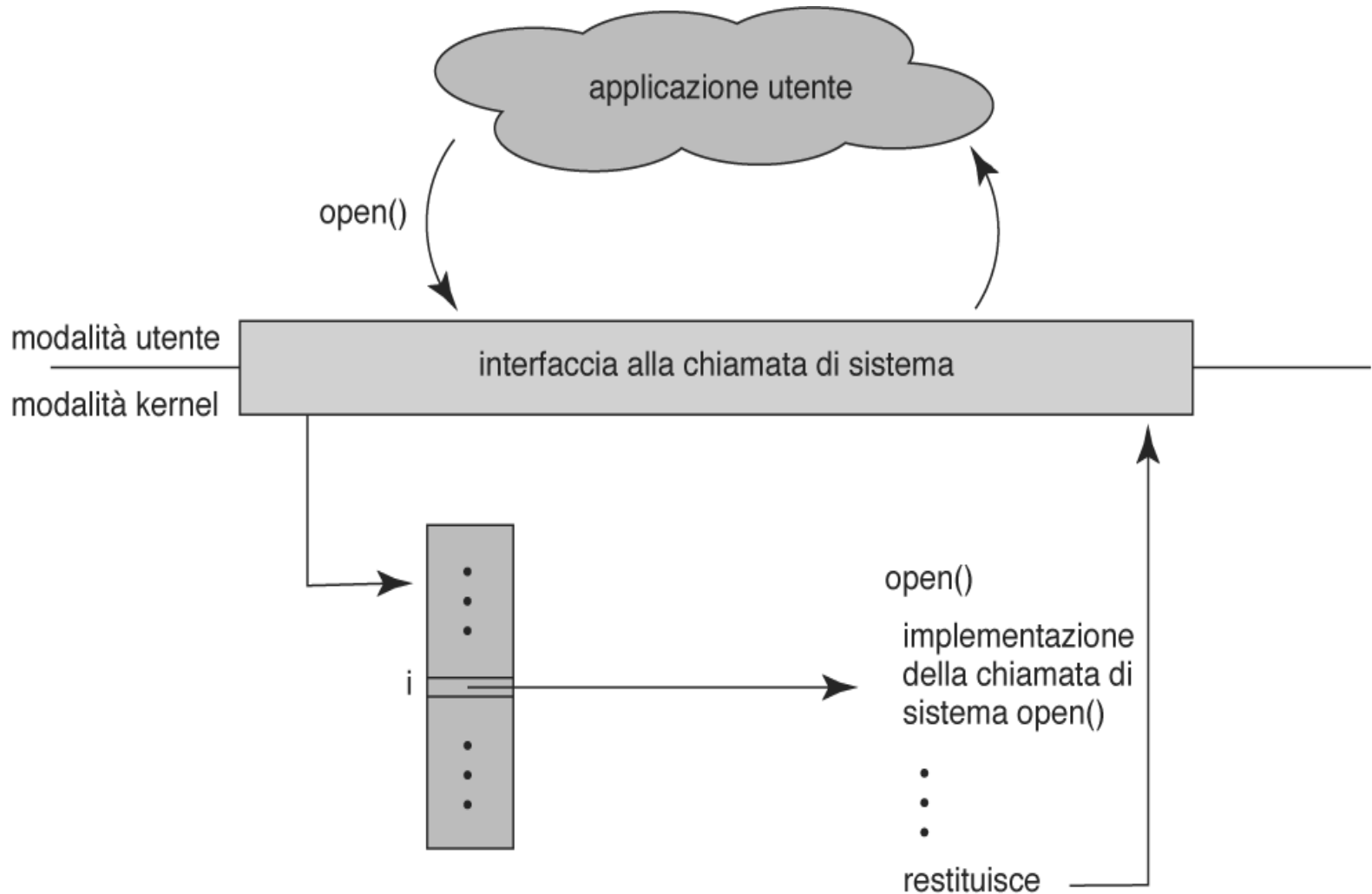
WINE. <https://www.winehq.org/about>

- Compatibility layer running on top of Posix-compliant Oses
- Open Source
- Wine translates Windows API calls into POSIX calls on-the-fly

# Implementazione delle System Call

- Tipicamente, un numero associato ad ogni system call
  - L' interfaccia alle chiamate di sistema mantiene una tabella indicizzata secondo questi numeri
- L' interfaccia invoca la system call usata, nel kernel del SO e poi ritorna lo stato della system call e gli eventuali valori di ritorno
- Il chiamante non ha necessità di conoscere nulla di come la system call è implementata

# open() chiamata da un programma utente



# Example of standard API: read()

```
#include <unistd.h>

ssize_t  read(int fd, void *buf, size_t count)
```

|              |               |            |
|--------------|---------------|------------|
| return value | function name | parameters |
|--------------|---------------|------------|

A program that uses the `read()` function must include the `unistd.h` header file, as this file defines the `ssize_t` and `size_t` data types (among other things). The parameters passed to `read()` are as follows:

- `int fd`—the file descriptor to be read
- `void *buf`—a buffer where the data will be read into
- `size_t count`—the maximum number of bytes to be read into the buffer

# System Call

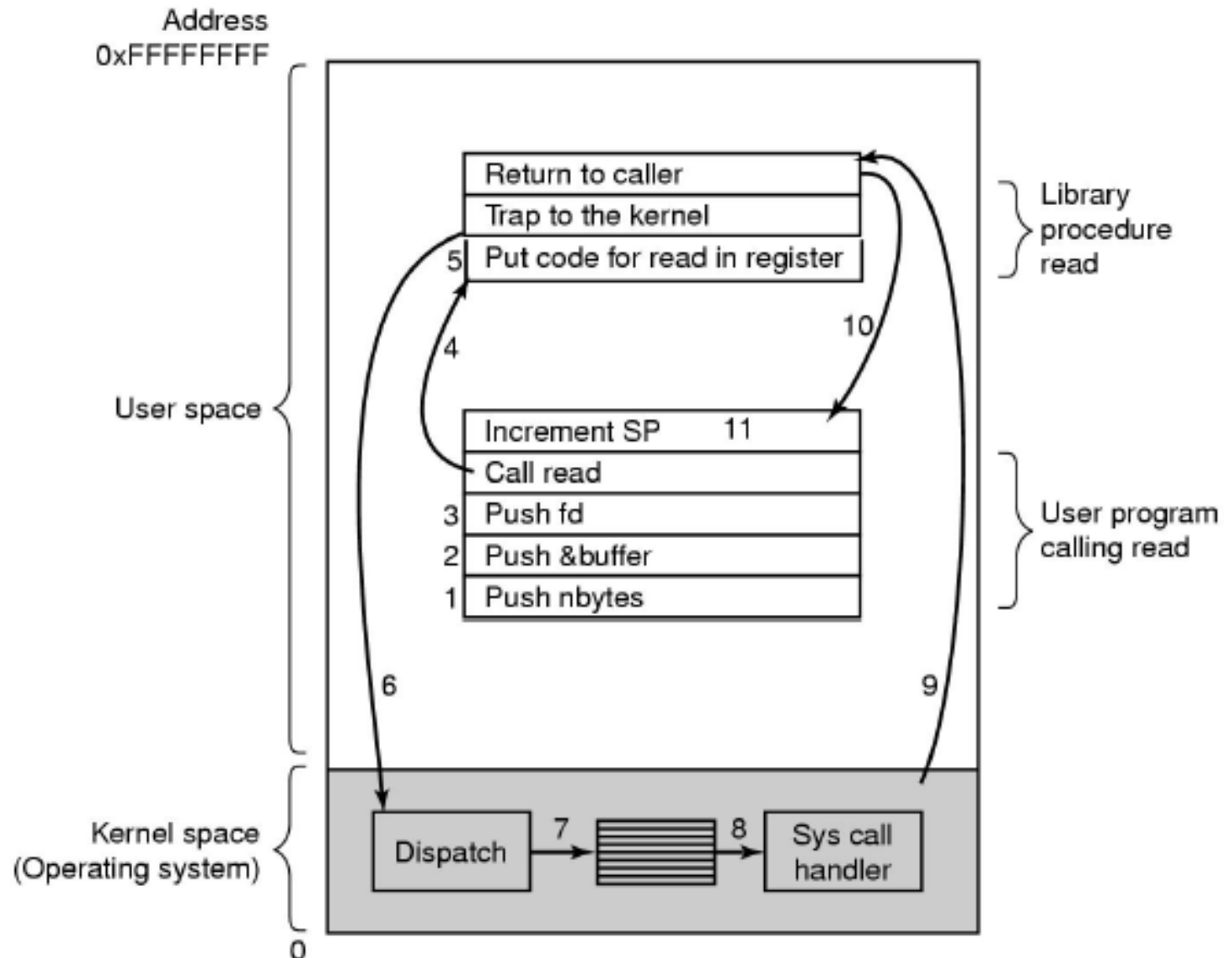
- Opzioni per la comunicazione tra il S.O. e un processo:

# System Call

- Opzioni per la comunicazione tra il S.O. e un processo:
  - Passare i parametri (della system call) tramite registri
  - Passare i parametri tramite lo stack del processo
  - Memorizzare i parametri in una tabella in memoria
    - L'indirizzo della tabella è passato in un registro o nello stack

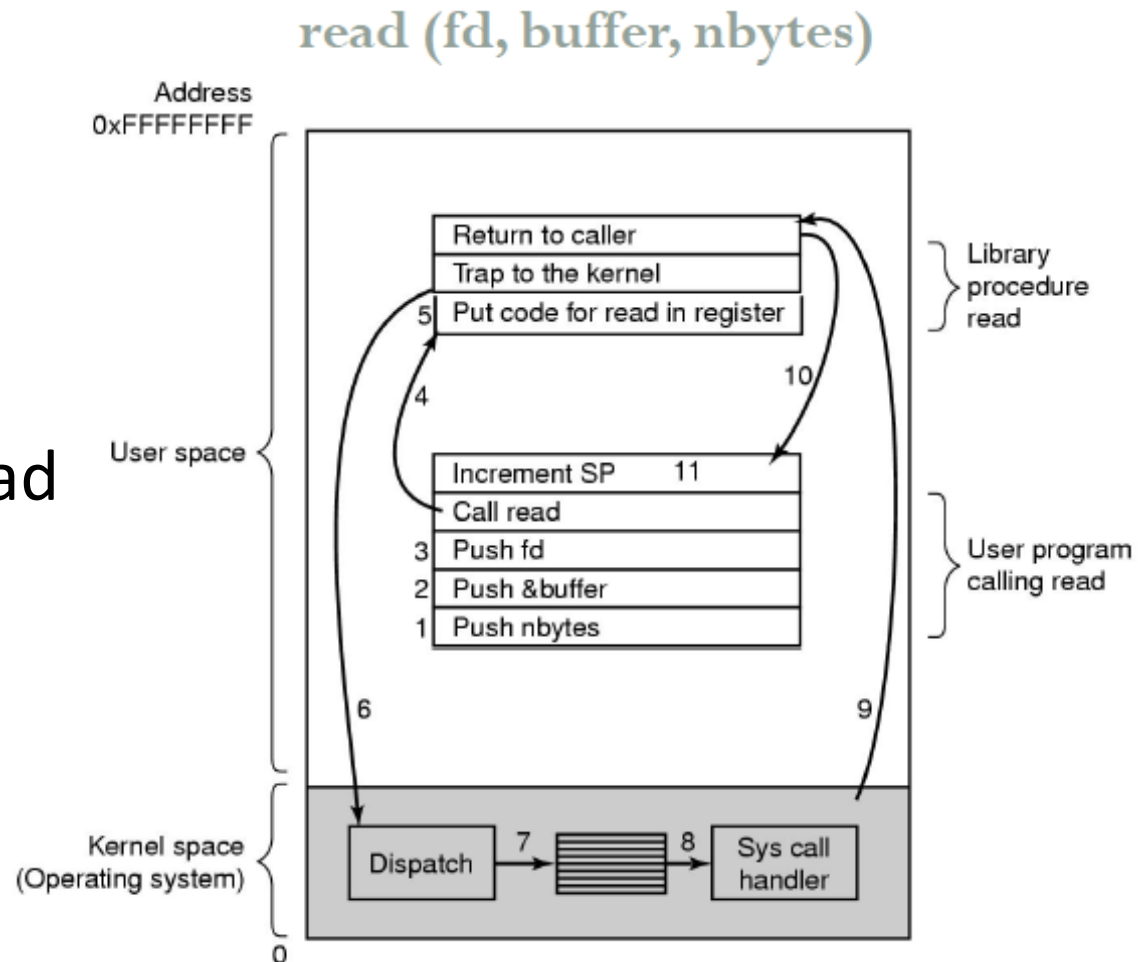
# Passaggio di parametri nello stack

`read (fd, buffer, nbytes)`



# Passaggio di parametri nello stack

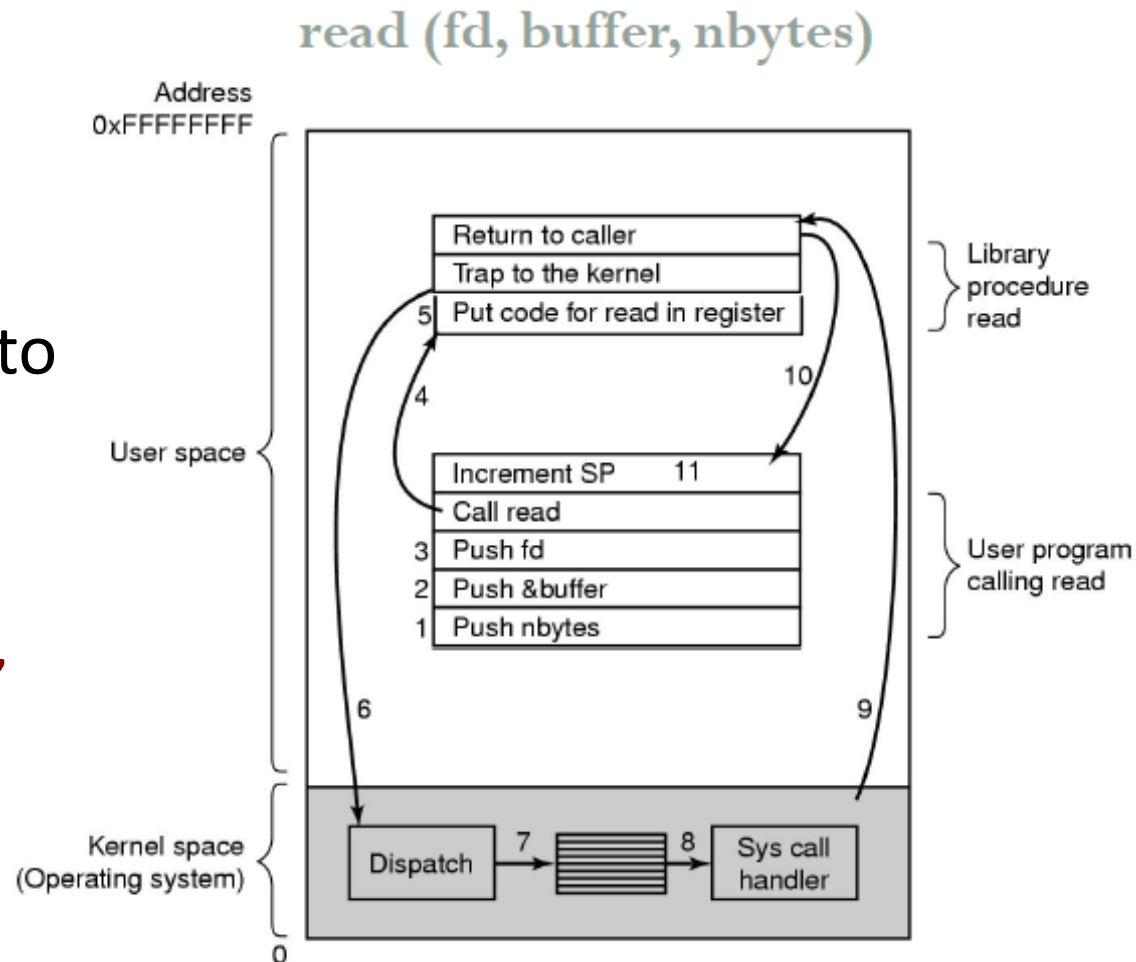
- 1-3. Salvataggio dei parametri sullo stack
- 4. Chiamata della funzione di libreria read





# Passaggio di parametri nello stack

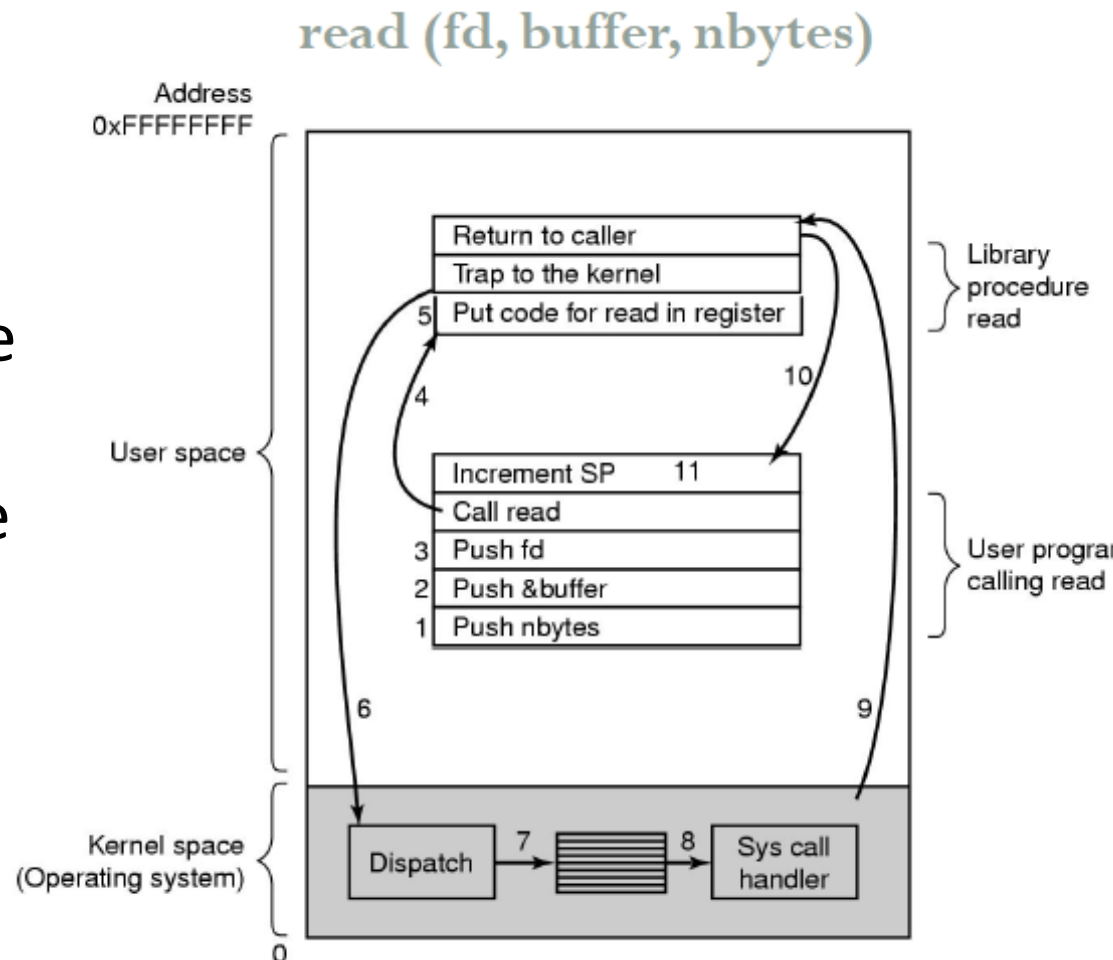
- 5. Caricamento del numero della system call in un registro fissato Rx
- 6. Esecuzione TRAP  
passaggio in kernel mode,  
salto al codice del dispatcher



# Passaggio di parametri nello stack

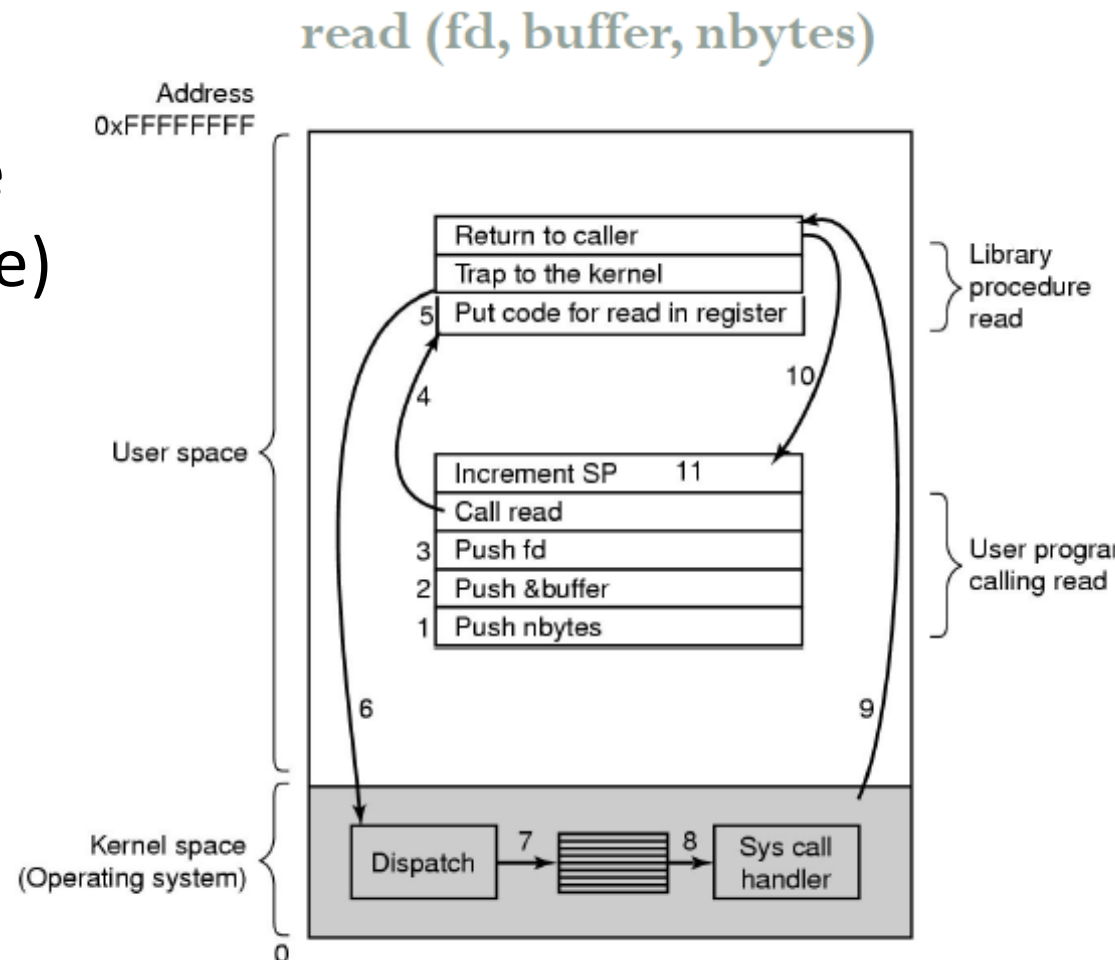
- 7-8. Selezione della system call secondo il codice in Rx ed invocazione del gestore appropriato
- 9. Ritorno alla funzione di libreria

ripristino user mode,  
caricamento del program  
counter PC



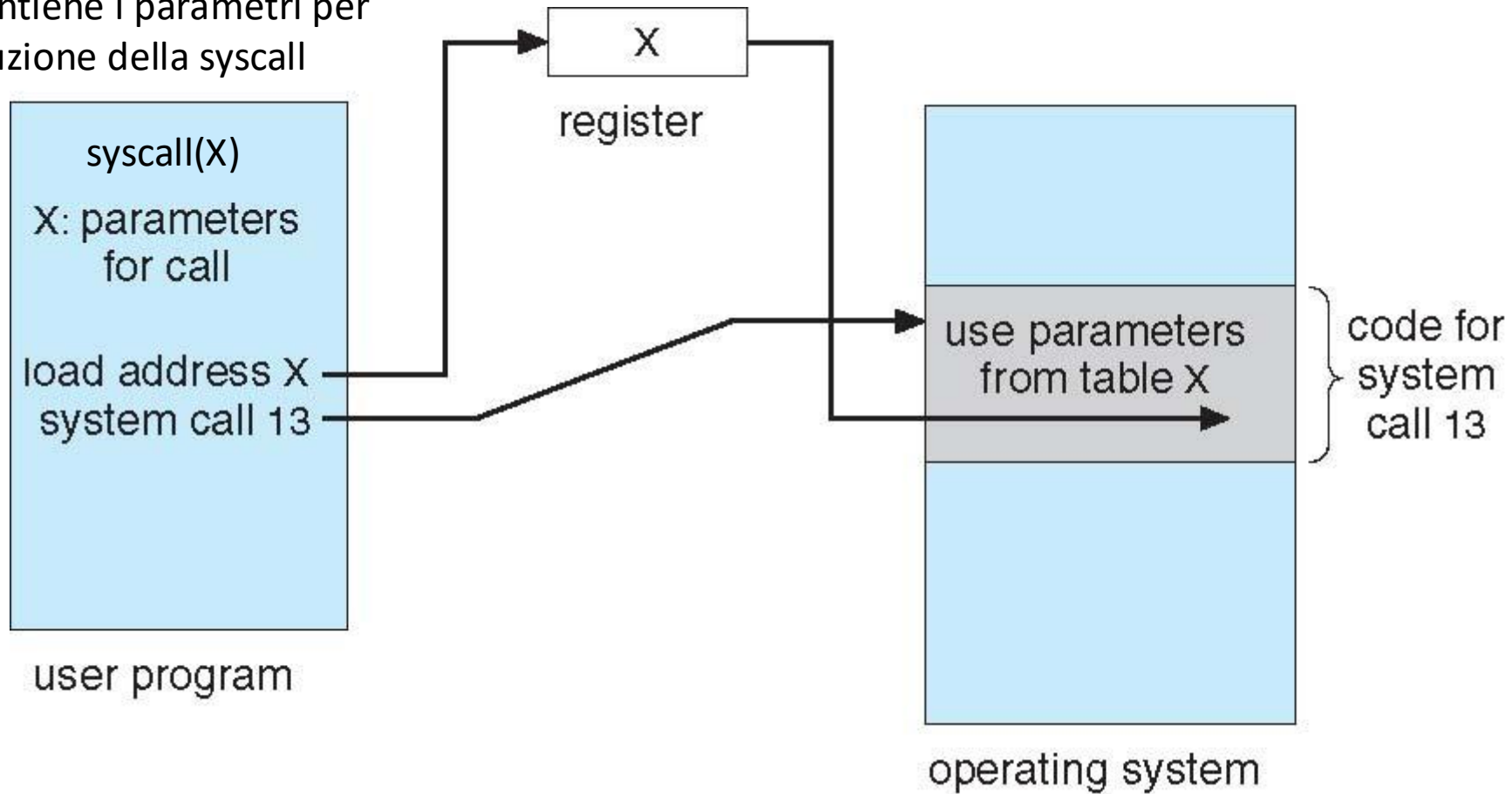
# Passaggio di parametri nello stack

- 10-11. Ritorno al codice utente (nel modo usuale) ed incremento dello stack pointer SP per rimuovere i parametri della chiamata a read



# Passaggio di parametri tramite tabella

X, parametro della syscall e'  
l'indirizzo della tabella, in mem,  
che contiene i parametri per  
l'esecuzione della syscall



# Riassumendo

- Servizi di un S.O.:
  - Esecuzione di programmi
  - Operazioni di I/O
  - Manipolazione del file system
  - Comunicazione
    - Memoria condivisa
    - Scambio di messaggi
  - Rilevamento di errori (logici e fisici)
  - Allocazione delle risorse
  - Contabilizzazione delle risorse
  - Protezione e sicurezza